

E commerce Sales Performance Analysis

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


# Set random seed for reproducibility

np.random.seed(42)


# 1. Generate synthetic e-commerce sales data

n_rows = 1000

categories = ['Electronics', 'Clothing', 'Home & Kitchen', 'Beauty', 'Sports']

dates = pd.date_range(start='2025-01-01', end='2025-12-31', freq='D')


data = {

    'Order_ID': [f'ORD{1000 + i}' for i in range(n_rows)],

    'Order_Date': np.random.choice(dates, size=n_rows),

    'Product_Category': np.random.choice(categories, size=n_rows, p=[0.3, 0.25, 0.2, 0.15, 0.1]),

    'Quantity': np.random.randint(1, 6, size=n_rows),

    'Unit_Price': np.random.uniform(10, 500, size=n_rows)

}


df = pd.DataFrame(data)

df['Sales'] = df['Quantity'] * df['Unit_Price']

df['Profit'] = df['Sales'] * np.random.uniform(0.1, 0.4, size=n_rows)
```

```
# Format floats
```

```
df['Unit_Price'] = df['Unit_Price'].round(2)
```

```
df['Sales'] = df['Sales'].round(2)
```

```
df['Profit'] = df['Profit'].round(2)
```

```
# Save to CSV as required
```

```
df.to_csv('ecommerce_sales_data.csv', index=False)
```

```
# 2. Inspect data as required by guidelines
```

```
print("--- Dataframe Head ---")
```

```
print(df.head())
```

```
print("\n--- Dataframe Info ---")
```

```
df.info()
```

```
# 3. Data Preparation for Visualizations
```

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
```

```
df['Month'] = df['Order_Date'].dt.to_period('M')
```

```
# Monthly Sales Summary (for line chart)
```

```
monthly_sales = df.groupby('Month')['Sales'].sum().reset_index()
```

```
monthly_sales['Month'] = monthly_sales['Month'].astype(str)
```

```
# Category Sales Summary (for bar chart) - sorted order as required
```

```
category_sales = df.groupby('Product_Category')['Sales'].sum().reset_index()
```

```
category_sales = category_sales.sort_values(by='Sales', ascending=False)
```

```
print("\n--- Monthly Sales Summary ---")  
  
print(monthly_sales)  
  
print("\n--- Category Sales Summary ---")  
  
print(category_sales)
```

4. Plotting

Configure default styles to prevent truncation and ensure readability

```
plt.rcParams["figure.figsize"] = (10, 6)  
  
plt.rcParams["figure.autolayout"] = True
```

Line Chart: Monthly Sales Trend

```
plt.clf() # Clear any existing plots  
  
plt.plot(monthly_sales['Month'], monthly_sales['Sales'], marker='o', color='b', linewidth=2)  
  
plt.title('Monthly Sales Performance Trend (2025)', fontsize=14, fontweight='bold')  
  
plt.xlabel('Month', fontsize=12)  
  
plt.ylabel('Total Sales ($)', fontsize=12)  
  
plt.grid(True, linestyle='--', alpha=0.6)  
  
plt.xticks(rotation=45)  
  
plt.savefig('monthly_sales_trend.png', dpi=300)
```

Bar Chart: Sales by Product Category (Sorted)

```
plt.clf() # Clear plot for the next chart  
  
bars = plt.bar(category_sales['Product_Category'], category_sales['Sales'], color='teal',  
edgecolor='black')  
  
plt.title('Sales Performance by Product Category', fontsize=14, fontweight='bold')  
  
plt.xlabel('Product Category', fontsize=12)  
  
plt.ylabel('Total Sales ($)', fontsize=12)  
  
plt.grid(axis='y', linestyle='--', alpha=0.6)
```

```
# Adding value labels on top of the bars for clarity

for bar in bars:

    yval = bar.get_height()

    plt.text(bar.get_x() + bar.get_width()/2, yval + 2000, f"${yval:,.2f}", ha='center', va='bottom',
    fontsize=10)

plt.savefig('category_sales_performance.png', dpi=300)
```